

Documentation technique

Choix technologiques, environnement, modélisation, sécurité et déploiement

ECF — Titre professionnel Développeur Web et Web Mobile
Version 1.0 — octobre 2026
Auteur : Romain Frigolet (FastDev)

Sommaire

1. Réflexions initiales technologiques
2. Configuration de l'environnement de travail
3. Architecture de l'application
4. Modèle conceptuel de données et base NoSQL
5. Diagramme de cas d'utilisation
6. Diagrammes de séquence
7. Sécurité
8. Accessibilité (RGAA) et RGPD
9. Tests réalisés
0. Documentation du déploiement

1. Réflexions initiales technologiques

Le sujet impose une base de données relationnelle et une base non relationnelle ; le reste de la stack est libre. Les critères de choix ont été : **la maîtrise et la justification de chaque ligne de code** (contexte d'examen), la **simplicité de déploiement** à coût nul, la **sécurité** et la **pérennité** pour un client sans compétence informatique.

1.1 Back-end : PHP 8.3 natif (sans framework) avec PDO

Option	Avantages	Inconvénients	Décision
Symfony / Laravel	Sécurité intégrée, ORM, productivité	Beaucoup de « magie » (ORM, migrations) qui masque le SQL ; l'énoncé précise que les fixtures/migrations ne prouvent pas la maîtrise du SQL	Écarté
Node.js / Express	Un seul langage front/back	Hébergement PHP plus répandu et moins cher ; écosystème plus éclaté pour un petit projet	Écarté
PHP 8.3 natif + PDO	Contrôle total, SQL écrit à la main, aucune dépendance lourde, hébergement universel, typage strict	Il faut implémenter soi-même routeur, CSRF, validation	**Retenu**

Pour compenser l'absence de framework, l'application suit une **architecture MVC** claire (contrôleur frontal, routeur, contrôleurs, repositories, services, vues) et réimplémente explicitement les protections qu'un framework fournirait (CSRF, échappement, sessions sécurisées). Une seule dépendance Composer : **PHPMailer** pour l'envoi SMTP fiable.

1.2 Bases de données

- **MySQL 8 (relationnel)** — source de vérité : intégrité référentielle (clés étrangères), transactions ACID (commande + décrément du stock), contraintes `CHECK`. Choisi plutôt que PostgreSQL pour sa disponibilité chez tous les hébergeurs mutualisés ; le code reste compatible MariaDB.
- **MongoDB 7 (NoSQL)** — imposé pour les statistiques de l'administrateur. Chaque commande y est copiée sous forme de **document dénormalisé** (titre du menu, montants, statut, date). Le calcul du nombre de commandes et du chiffre d'affaires par menu est fait par le **pipeline d'agrégation** de MongoDB, sans jointure et sans charger la base transactionnelle. Accès via l'extension native `ext-mongodb` (pas de surcouche).

1.3 Front-end

- **HTML5 sémantique + Bootstrap 5.3** : grille responsive et composants accessibles éprouvés (navigation, alertes) ; personnalisé par variables CSS selon la charte. Tailwind a été écarté (classes utilitaires très verbeuses dans des gabarits PHP).
- **JavaScript natif (ES2020, `fetch`)** : filtres des menus sans rechargement, estimation du prix, galerie. Un framework (React, Vue) serait disproportionné pour ces interactions ; le site reste utilisable sans JavaScript (amélioration progressive).
- **Chart.js 4** : graphique comparatif de l'administrateur (léger, accessible via un tableau de données équivalent).
- Toutes les ressources (Bootstrap, Chart.js, polices) sont **auto-hébergées** : aucune dépendance à un CDN, ce qui permet une CSP stricte et évite tout transfert de données personnelles (RGPD).

1.4 Déploiement

L'application est conteneurisée avec **Docker** (image PHP 8.3 + Apache) : le même environnement sert en développement et en production. Hébergement retenu, gratuit et adapté : **Render** (application), **Aiven** (MySQL

managé), **MongoDB Atlas** (cluster M0) et **Brevo** (SMTP transactionnel). Voir la partie 10.

2. Configuration de l'environnement de travail

Outil	Usage	Justification
Visual Studio Code	Éditeur	Extensions PHP Intelephense, ESLint, Prettier, GitLens, Docker, Live Share
PHP 8.3 + Composer 2	Langage et dépendances	Version maintenue, typage strict, autoload PSR-4
Docker Desktop + Docker Compose	Environnement local complet	Un seul `docker compose up` lance PHP/Apache, MySQL 8, MongoDB 7 et Mailpit : environnement identique pour tous et identique à la production
Mailpit	Capture des e-mails en local	Tester tous les e-mails sans envoyer de vrais messages
MySQL Workbench / MongoDB Compass	Clients de base de données	Visualiser les tables et les documents, tester les requêtes
Git + GitHub	Versionnement	Branches `main` / `develop` / `feature/*`, intégration par Pull Requests, dépôt public
Navigateurs + DevTools, Lighthouse, WAVE	Tests et audit	Responsive, réseau, performances, accessibilité
GitHub Projects	Gestion de projet	Tableau Kanban (voir document « Gestion de projet »)

2.1 Mise en route

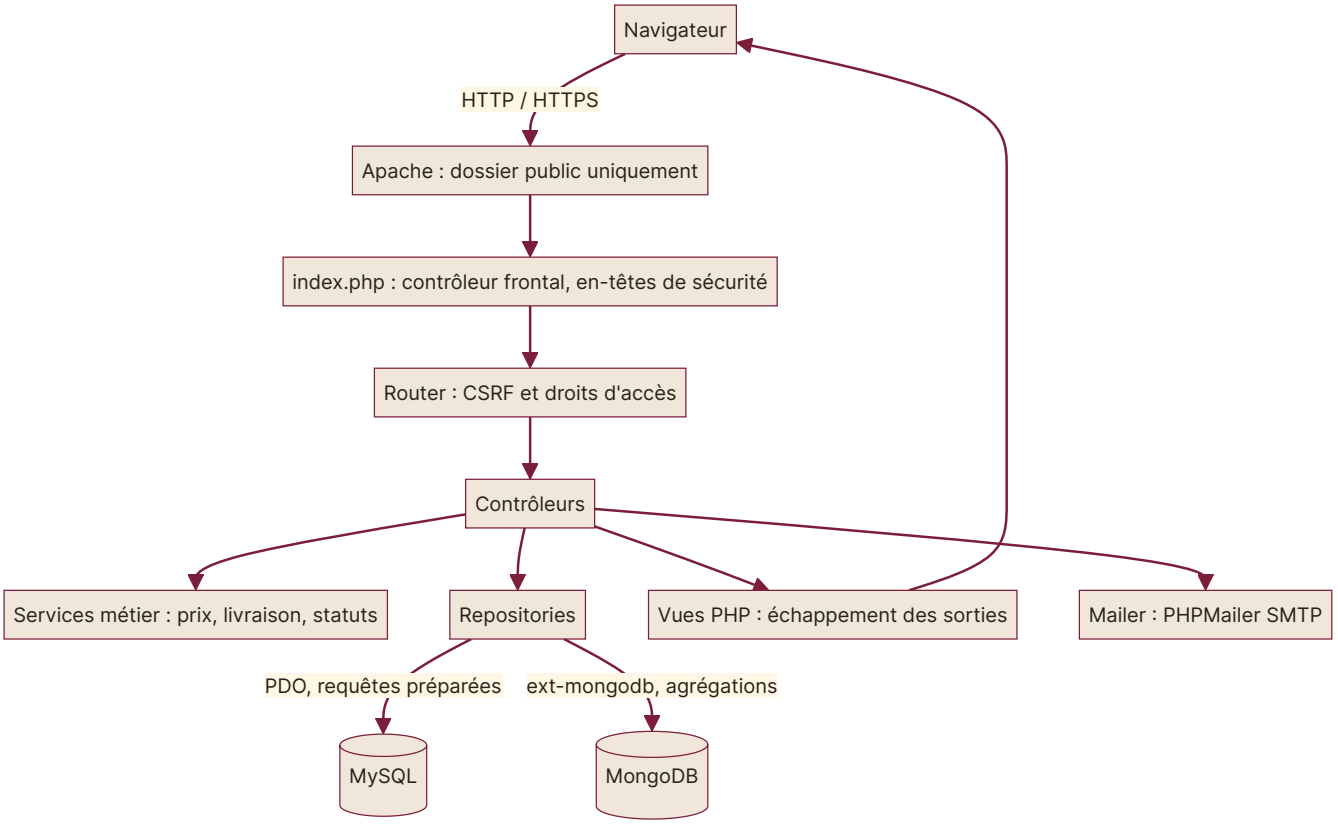
```
git clone https://github.com/Romainfrigolet/vite-et-gourmand.git
cd vite-et-gourmand
docker compose up -d --build
# Application : http://localhost:8080   –   E-mails (Mailpit) : http://localhost:8025
```

Au premier démarrage, le conteneur MySQL exécute `database/schema.sql` puis `database/data.sql` ; le conteneur applicatif reconstruit ensuite la collection MongoDB (`bin/sync-stats.php`). Une installation sans Docker (PHP + MySQL + MongoDB locaux, serveur `php -S`) est décrite dans le `README.md` .

2.2 Configuration par variables d'environnement

Aucun secret n'est écrit dans le code. Le fichier `.env` (ignoré par Git) contient les accès aux bases et au SMTP ; `.env.example` sert de modèle. En production, ces variables sont saisies dans l'interface de l'hébergeur.

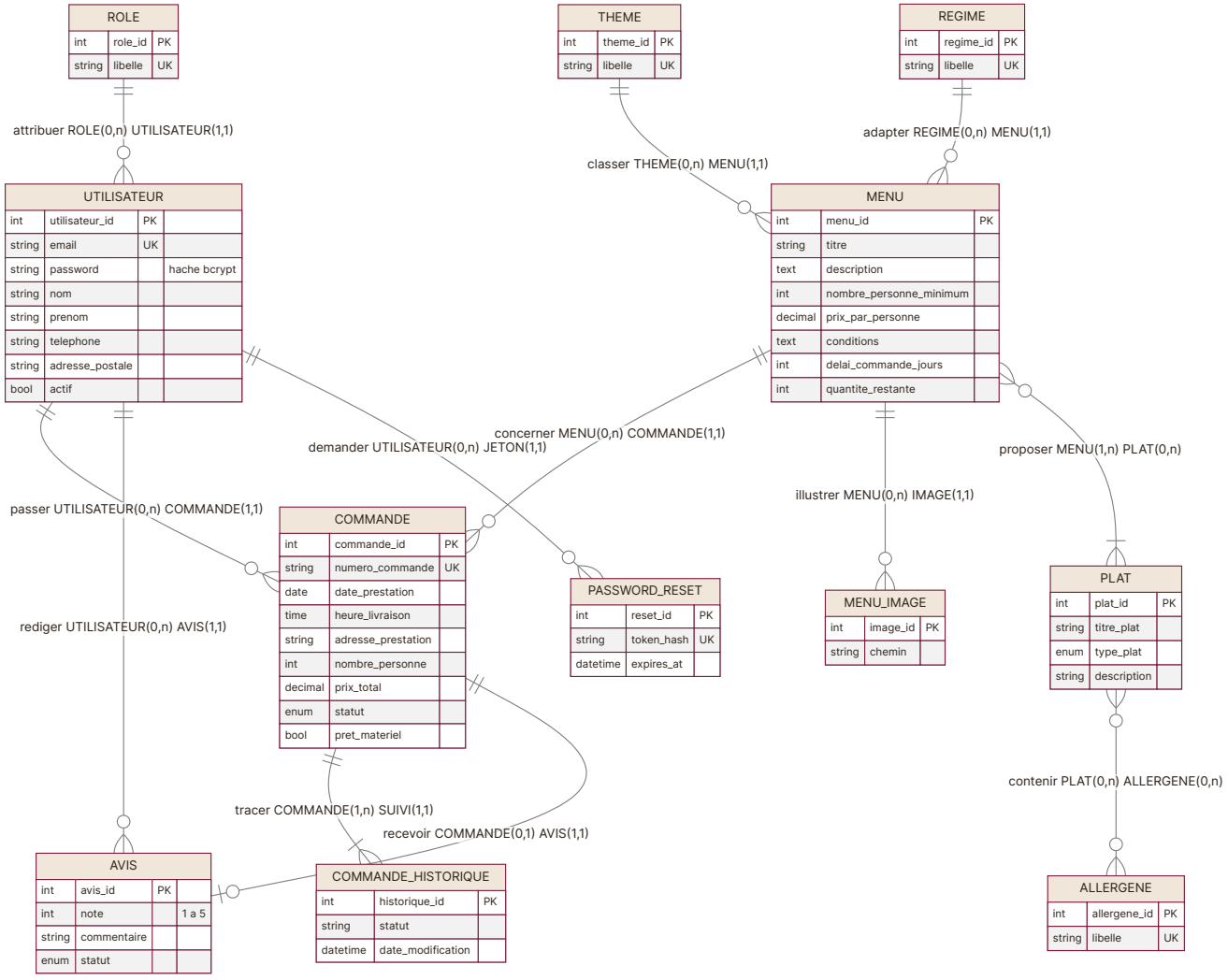
3. Architecture de l'application



- **Contrôleur frontal** (`public/index.php`) : point d'entrée unique ; seul `public/` est exposé, le code source et le `.env` sont inaccessibles depuis le web.
- **Routeur** (`src/Core/Router.php`) : associe méthode + URL à un contrôleur, vérifie le jeton CSRF des requêtes POST et applique les middlewares d'accès (`guest`, `auth`, `customer`, `staff`, `admin`).
- **Repositories** : seule couche qui parle aux bases ; toutes les requêtes SQL sont préparées.
- **Services** : règles métier testables isolément (`PricingService`, `DeliveryService`, `OrderStatus`, `OrderForm`).

4. Modèle conceptuel de données et base NoSQL

Le modèle reprend le MCD fourni en annexe de l'énoncé et l'enrichit : historique des statuts (suivi de commande), galerie d'images, délai de commande, jetons de réinitialisation, journal des tentatives de connexion. Les **cardinalités** sont indiquées selon la notation Merise *(min,max)* sur chaque association : la cardinalité est lue du côté de l'entité qu'elle suit (ex. « UTILISATEUR (0,n) passe (1,1) COMMANDE » : un utilisateur passe 0 à n commandes, une commande est passée par 1 et 1 seul utilisateur).



Cardinalités des associations

Entité A	Card. A	Association	Card. B	Entité B	Traduction dans le MPD
ROLE	0,n	attribuer	1,1	UTILISATEUR	clé étrangère `utilisateur.role_id` NOT NULL
UTILISATEUR	0,n	passer	1,1	COMMANDE	`commande.utilisateur_id` NOT NULL
UTILISATEUR	0,n	rédiger	1,1	AVIS	`avis.utilisateur_id` NOT NULL
UTILISATEUR	0,n	demander	1,1	PASSWORD_RESET	`password_reset.utilisateur_id` (suppression en cascade)
MENU	0,n	concerner	1,1	COMMANDE	`commande.menu_id` NOT NULL

THEME	0,n	classer	1,1	MENU	`menu.theme_id` NOT NULL
REGIME	0,n	adapter	1,1	MENU	`menu.regime_id` NOT NULL
MENU	0,n	illustrer	1,1	MENU_IMAGE	`menu_image.menu_id` (cascade)
MENU	1,n	proposer	0,n	PLAT	table d'association `menu_plat` ; « au moins un plat » contrôlé par l'application
PLAT	0,n	contenir	0,n	ALLERGENE	table d'association `plat_allergene`
COMMANDE	1,n	tracer	1,1	COMMANDE_HISTORIQUE	`commande_historique.commande_id` ; une ligne créée dès la commande
COMMANDE	0,1	recevoir	1,1	AVIS	`avis.commande_id` NOT NULL + `UNIQUE`

Tables indépendantes (sans association) : `horaire`, `contact_message`, `login_attempt`. Le script complet (17 tables, clés étrangères, index, contraintes CHECK) est dans `database/schema.sql`, les données dans `database/data.sql`.

4.1 Règles de gestion traduites dans le modèle

- Un plat peut appartenir à plusieurs menus → table d'association `menu_plat` (n,n).
- Un plat possède une liste d'allergènes → `plat_allergene` (n,n) vers les 14 allergènes réglementaires.
- Une commande reçoit au plus un avis → contrainte `UNIQUE (commande_id)` sur `avis`.
- Les prix sont **figés dans la commande** (prix menu, remise, livraison, total) : une modification ultérieure du menu ne change pas l'historique.
- Un menu déjà commandé n'est jamais supprimé physiquement mais désactivé (`actif = 0`).
- Le stock est décrémenté par `UPDATE … WHERE quantite_restante > 0` dans une transaction : impossible de vendre un menu épuisé, même en cas de commandes simultanées.

4.2 Base NoSQL : collection `commandes_stats`

```
{
  "commande_id": 2, "numero": "VG-2026-0002",
  "menu_id": 3, "menu_titre": "Déjeuner de Pâques en famille",
  "date_commande": ISODate("2026-03-20T17:40:00Z"), "date_prestation": "2026-04-05",
  "nombre_personne": 15, "prix_menu": 540.0, "remise": 54.0, "prix_livraison": 8.19,
  "prix_total": 494.19, "chiffre_affaires": 486.0, "statut": "terminee"
}
```

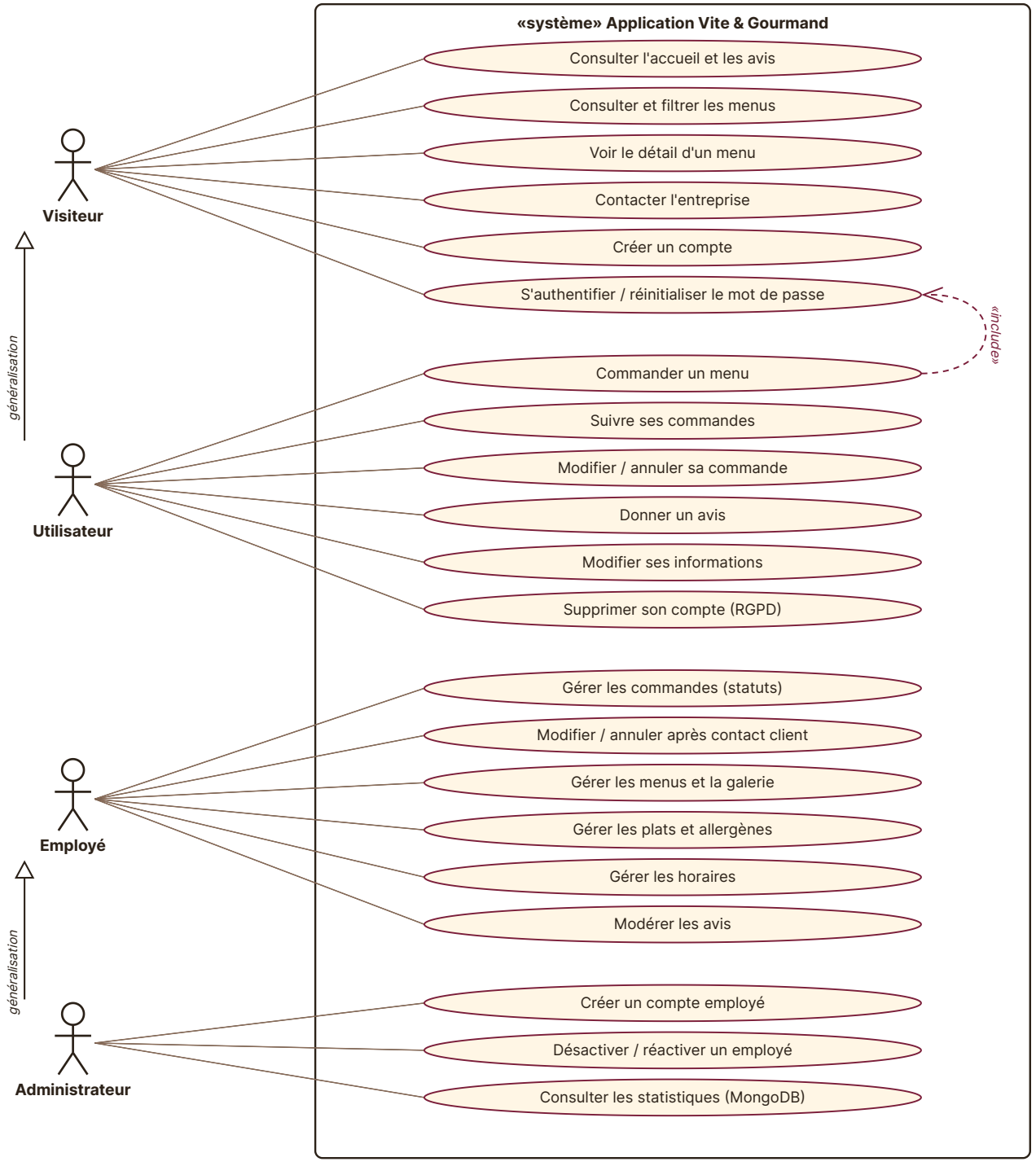
Pipeline d'agrégation utilisé par l'administrateur (filtres optionnels par menu et période) :

```
[
  { "$match": { "statut": { "$ne": "annulee" }, "menu_id": 3,
                "date_commande": { "$gte": ISODate("2026-01-01"), "$lte": ISODate("2026-12-31") }
} },
  { "$group": { "_id": "$menu_id", "nb_commandes": { "$sum": 1 },
                "chiffre_affaires": { "$sum": "$chiffre_affaires" }, "total_encaisse": { "$sum":
"$prix_total" } } },
  { "$sort": { "nb_commandes": -1 } }
]
```

MySQL reste la source de vérité : chaque création ou modification de commande déclenche `StatsRepository::sync()` (upsert du document). Le script `bin/sync-stats.php` reconstruit toute la collection si besoin. Une indisponibilité de MongoDB n'empêche jamais une commande d'aboutir (l'erreur est journalisée).

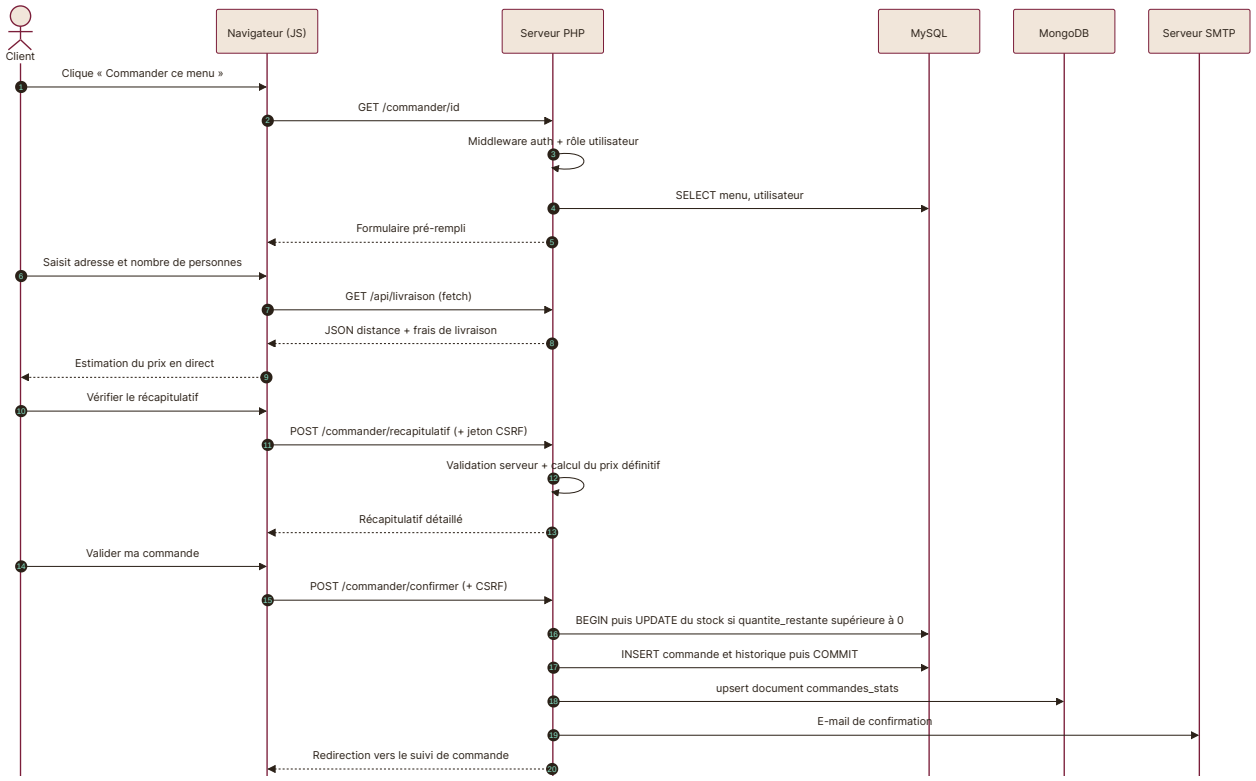
5. Diagramme de cas d'utilisation

Diagramme UML : les acteurs sont à l'extérieur de la frontière du système, chaque cas d'utilisation est une ellipse reliée à l'acteur qui le déclenche. La **généralisation** (flèche à pointe creuse) indique qu'un Utilisateur hérite des cas du Visiteur et l'Administrateur de ceux de l'Employé. La relation **«include»** indique que commander un menu nécessite d'être authentifié.

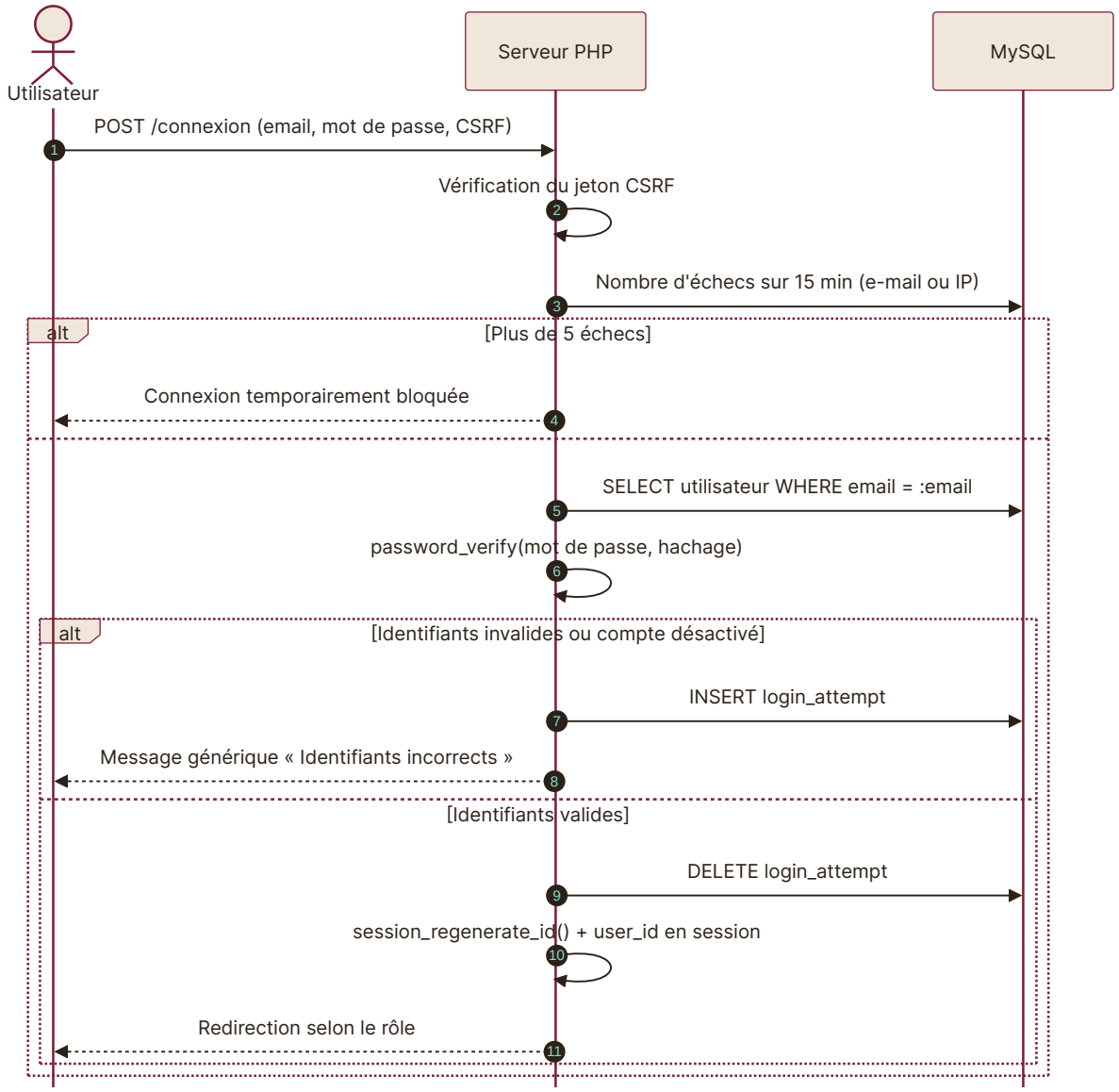


6. Diagrammes de séquence

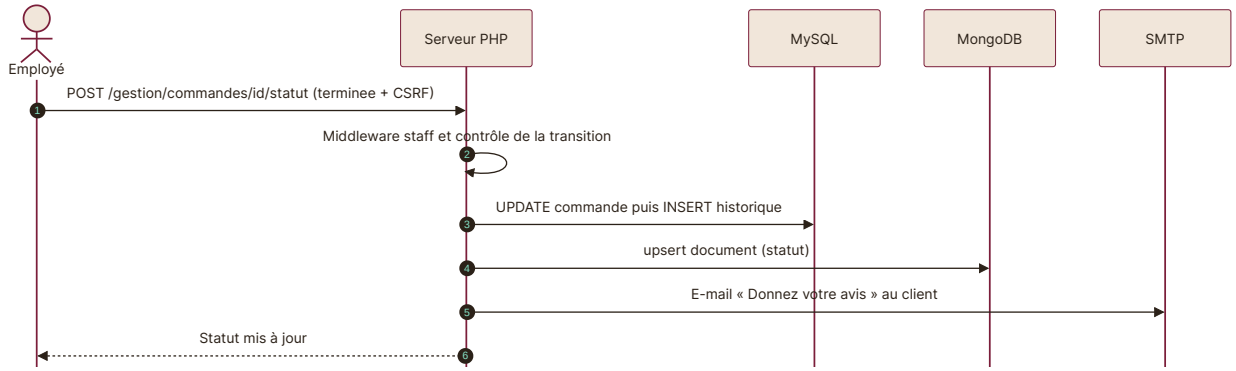
6.1 Commande d'un menu



6.2 Connexion sécurisée



6.3 Passage d'une commande au statut « terminée » par un employé



7. Sécurité

Menace	Contre-mesure mise en place
Injection SQL	PDO avec requêtes préparées partout, `ATTR_EMULATE_PREPARES = false` ; filtres dynamiques construits uniquement avec des paramètres liés ; utilisateur MySQL applicatif limité à SELECT/INSERT/UPDATE/DELETE
XSS	Toute donnée affichée passe par `e()` (`htmlspecialchars`, ENT_QUOTES, UTF-8) ; en-tête **Content-Security-Policy** stricte (`script-src 'self'`, aucun script ou style inline)
CSRF	Jeton aléatoire (32 octets) en session, vérifié par le routeur sur chaque POST avec `hash_equals` ; cookie `SameSite=Lax`
Vol de mot de passe	`password_hash` (bcrypt) + `password_needs_rehash` ; politique imposée côté client et serveur ; changement de mot de passe soumis au mot de passe actuel
Force brute	Table `login_attempt` : blocage après 5 échecs en 15 min par e-mail (10 par IP) ; comparaison à temps constant même si le compte n'existe pas
Énumération de comptes	Messages génériques à la connexion et au « mot de passe oublié »
Détournement de session	Cookie `HttpOnly`, `Secure` en HTTPS, `use_strict_mode`, régénération de l'ID à la connexion, expiration après 2 h d'inactivité ; compte désactivé = déconnexion immédiate
Réinitialisation détournée	Jeton de 256 bits, seul son SHA-256 est stocké, validité 1 h, usage unique
Contrôle d'accès défaillant / IDOR	Middlewares par rôle sur chaque route ; une commande n'est accessible à un client que si `utilisateur_id` correspond ; aucun compte administrateur créable depuis l'application
Upload malveillant	Type MIME vérifié sur le contenu (`finfo` + `getimagesize`), 2 Mo max., nom aléatoire, moteur PHP désactivé dans `uploads/`
Clickjacking, sniffing	`X-Frame-Options: DENY`, `frame-ancestors 'none'`, `X-Content-Type-Options: nosniff`, HSTS
Fuite d'informations	Erreurs techniques jamais affichées en production (journalisées), `expose_php = Off`, secrets dans des variables d'environnement
Redirection ouverte	Redirection post-connexion limitée aux chemins internes
Manipulation du prix	Le prix est toujours recalculé côté serveur ; le calcul JavaScript n'est qu'indicatif
Spam du formulaire de contact	Champ « piège » invisible (honeypot)

8. Accessibilité (RGAA) et RGPD

8.1 Accessibilité

- Langue déclarée (`lang="fr"`), titre de page unique, structure de titres hiérarchique, landmarks (`header`, `nav`, `main`, `footer`).
- Lien d'évitement « Aller au contenu principal », navigation complète au clavier, focus visible (contour or 3 px).
- Chaque champ a un `label` associé ; erreurs reliées au champ (`aria-describedby`, `aria-invalid`) ; groupes de boutons radio en `fieldset/legend`.
- Résultats des filtres et estimation du prix annoncés aux lecteurs d'écran (`aria-live`).

- Contrastes AA vérifiés pour toutes les couleurs de texte ; information jamais transmise par la seule couleur (badges avec libellé).
- Images informatives avec texte alternatif (obligatoire à l'upload) ; images décoratives avec `alt=""`.
- Graphique doublé d'un tableau de données ; animations désactivées si `prefers-reduced-motion`.

8.2 RGPD

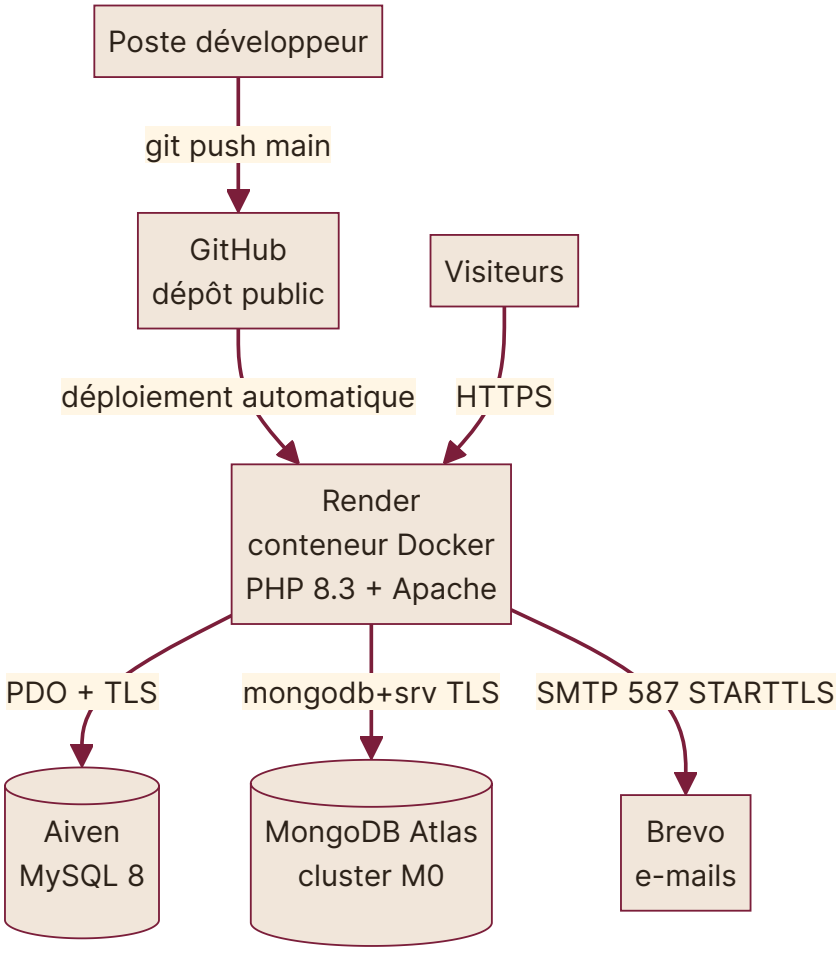
- Consentement explicite à la politique de confidentialité lors de l'inscription ; minimisation des données collectées.
- Droit de rectification (« Mes informations ») et droit à l'effacement (suppression du compte = anonymisation, commandes conservées pour la comptabilité).
- Aucun cookie tiers, aucun traceur, polices et librairies auto-hébergées : pas de bandeau de consentement nécessaire (seul cookie de session strictement nécessaire).
- Pages Mentions légales, CGV et Politique de confidentialité accessibles depuis le pied de page.

9. Tests réalisés

Les parcours sont testés de bout en bout par le script `tests/smoke.sh`, exécuté automatiquement par **GitHub Actions** à chaque push : l'intégration continue construit l'image Docker, démarre MySQL, MongoDB et Mailpit, puis vérifie 38 points (pages, sécurité, inscription, commande, statuts, statistiques MongoDB, e-mails). L'affichage a été vérifié en bureau et en mobile avec un navigateur automatisé (Playwright).

Scénario	Résultat attendu	Statut
Inscription avec mot de passe faible	Refus avec message explicite	✔
Inscription valide	Compte « utilisateur » + e-mail de bienvenue	✔
Connexion d'un compte désactivé	Refus	✔
Réinitialisation du mot de passe, puis réutilisation du lien	Succès, puis lien refusé	✔
POST sans jeton CSRF	HTTP 403	✔
Filtres des menus (thème, régime, prix, personnes)	Résultats mis à jour sans rechargement	✔
Commande sous le minimum de personnes / avant le délai	Refus avec message	✔
Commande 10 pers. (min. 4) livrée à Pessac	380 € − 38 € + 8,19 € = 350,19 €	✔
Accès à la commande d'un autre client	HTTP 404	✔
Client sur /gestion, employé sur /admin	HTTP 403	✔
Saut de statut (accepté → livré)	Refusé	✔
Annulation employé sans motif ni mode de contact	Refusée	✔
Upload d'un fichier PHP renommé en image	Refusé	✔
Statistiques MongoDB comparées aux totaux SQL	Montants identiques	✔
Console navigateur (CSP)	Aucune erreur	✔
E-mails de bienvenue, confirmation, invitation à l'avis (Mailpit)	Reçus	✔

10. Documentation du déploiement



10.1 Démarche

Le client exige une application en ligne et fonctionnelle à la livraison. J'ai choisi des services **managés et gratuits**, chacun spécialisé, plutôt qu'un serveur à administrer : HTTPS, sauvegardes et mises à jour de sécurité sont assurés par les hébergeurs. L'image Docker garantit que la production exécute exactement le même environnement que le poste de développement.

10.2 Étapes

1. **Base MySQL (Aiven)** : créer un compte sur aiven.io → *Create service* → MySQL, plan *Free*. Noter l'hôte, le port, l'utilisateur et le mot de passe, et copier le certificat *CA*. Les tables et les données sont créées automatiquement au premier démarrage de l'application (script `bin/install-db.php --if-empty`, activé par `DB_AUTO_INSTALL=true`) ; l'import peut aussi être lancé depuis un poste avec `php bin/install-db.php`.
2. **MongoDB (Atlas)** : créer un cluster gratuit M0, un utilisateur de base de données et autoriser l'accès réseau (*Network Access* → 0.0.0.0/0, car l'IP de Render n'est pas fixe sur l'offre gratuite). Copier l'URI `mongodb+srv://…`.
3. **SMTP (Brevo)** : créer un compte, valider l'adresse expéditrice, générer une clé SMTP (*SMTP & API*). Hôte `smtp-relay.brevo.com`, port 587.
4. **Application (Render)** : le dépôt contient un *Blueprint* (`render.yaml`) qui décrit le service : image Docker, offre gratuite, branche `main`, variables non secrètes déjà renseignées (`APP_ENV=production`, `APP_DEBUG=false`, SMTP Brevo…). *New* → *Blueprint* → choisir le dépôt : Render demande uniquement les valeurs secrètes (`DB_HOST`, `DB_PORT`, `DB_USER`, `DB_PASSWORD`, `DB_SSL_CA` = contenu du certificat Aiven,

`MONGO_URI`, `MAIL_USERNAME`, `MAIL_PASSWORD`, `MAIL_FROM`, `MAIL_CONTACT`). L'adresse publique est fournie automatiquement par Render (`RENDER_EXTERNAL_URL`).

5. Render construit l'image (`Dockerfile`) puis démarre le conteneur : le script `docker/entrypoint.sh` adapte Apache au port imposé (`$PORT`) et synchronise les statistiques MongoDB depuis MySQL.
6. **Vérifications** : page d'accueil en HTTPS, connexion des quatre comptes de test, passage d'une commande, réception de l'e-mail, statistiques de l'administrateur.
7. **Mises à jour** : toute fusion de `develop` dans `main` suivie d'un `git push` redéploie automatiquement l'application.

> Limites des offres gratuites : Render met le service en veille après 15 minutes sans visite (premier chargement ≈ 1 minute) et son disque n'est pas persistant (les images téléversées sont perdues à chaque redéploiement) ; Aiven peut éteindre une base gratuite restée inactive. Pour garder l'application accessible, le workflow GitHub Actions `.github/workflows/keepalive.yml` interroge toutes les 10 minutes une page qui lit la base : l'application reste réveillée et la base garde une activité. En production réelle, il faudrait passer à des offres payantes (instance toujours active, disque persistant ou stockage objet S3 pour les images).

10.3 Bonnes pratiques de production appliquées

- HTTPS forcé par l'hébergeur + HSTS ; cookies `Secure`.
- Mots de passe des comptes de démonstration à changer après la soutenance.
- Connexions chiffrées vers MySQL (TLS avec CA) et MongoDB (`mongodb+srv`).
- Journalisation des erreurs dans `storage/logs` et dans les logs Render, sans affichage à l'écran.